

Contents

My project — walkthrough notes	1
PART 1 — Literature review, problem and objectives	1
PART 2 — Dataset, EDA and models	3
PART 3 — The paper	7
What to show sir, in order	7
Numbers I must remember	7
Questions sir may ask, and my answers	8
Words I should be able to explain in one line	8

My project — walkthrough notes

My own notes for explaining the project. Written so I can read straight off it.

Project: Predicting if a diabetic patient comes back to hospital within 30 days of going home.

Paper title: Beyond the Encounter-Level Benchmark: Patient-Level Validation, Imbalance Corrections and the Provenance of Signal in 30-Day Hospital Readmission Prediction.

The three things I was asked to do

1. Do a literature review, find the research gaps, and write a clear research problem with objectives.
2. Pick a dataset and justify it. Clean it, do EDA, do feature engineering. Build, compare and evaluate ML models. Explain the findings and link them back to the literature.
3. Present everything as a research paper in the Springer LNCS format, with the code and the dataset details.

Below is what I did for each one.

PART 1 — Literature review, problem and objectives

1.1 How I searched for papers (this is my Colab code)

I did not search by hand. I wrote a notebook that does the search automatically, so the whole thing is repeatable and logged. **File to open:** `lit/lit_search.ipynb`.

What the notebook does, step by step:

1. **I write my inclusion and exclusion criteria first, before searching.** This is the important bit. If I write the criteria after seeing the results, I am just picking the papers I like. Writing them first stops that.
 - Include: adult hospital patients, structured electronic health records, papers about predicting readmission, using ML or statistical models on tabular data, published 2015 or later.

- Exclude: reviews with no model of their own, wrong outcome, image or genomics data, and papers that report no performance numbers.
2. **It calls the OpenAlex API** with four search queries. OpenAlex is a free academic database, no sign-up needed.
 3. **It logs every query** — the exact words, the date and time, how many hits, how many I took. That log becomes Table 1 in my paper.
 4. **It removes duplicates** — the same paper can come up in more than one query.
 5. **It pulls out the details of each paper** — title, year, journal, DOI, abstract.
 6. **It writes out a refs.bib file** with the DOIs, so my citations are real and not made up.

The numbers from my search

Stage	Count
Retrieved from OpenAlex (4 queries $\times$ 25)	100
After removing duplicates	77
Screened against my criteria	77
Included in the review	23

Every one of the 77 papers, with my decision and my reason, is in **Appendix A of my paper (Table 10)**. Nothing is hidden.

If sir asks “why these four queries?”

Because the problem statement has four parts and I wanted one query per part: readmission + ML + EHR; readmission + missing data + class imbalance; patient characteristics + readmission; feature engineering + readmission.

1.2 What the literature says

The short version: **everyone gets roughly the same accuracy, and it is not very good.**

- Kansagara (2011), a review of readmission models: c-statistic around 0.60–0.65.
- Mahmoudi (2020) in the BMJ, reviewed 41 models built on hospital records: median AUROC around **0.68**. Also found that most papers never check calibration.
- Huang (2021), a scoping review of ML for readmission: tree models win, but only by a small amount.

On my exact dataset:

- **Strack (2014)** is the paper that released the dataset. They found that measuring HbA1c is linked to lower readmission.
- Shang (2021), Lu & Uddin (2022), Liu (2024) all built ML models on it. Random forest and boosting come out on top.

1.3 The three research gaps I found

Gap 1 — They split the data wrongly. The dataset has 101,766 hospital visits but only 71,518 patients. So one patient can appear many times. If you split the data randomly, the same patient ends up in both the training set and the test set. The model can then “recognise” the patient instead of learning about risk. Nobody had measured how much this inflates the results.

Gap 2 — They fix the class imbalance without checking what it does to the probabilities. Only 11% of visits end in readmission. Almost everyone applies SMOTE or undersampling. But two papers outside this dataset (van den Goorbergh 2022, Welvaars 2023) show this makes the predicted probabilities far too high. Nobody had tested that on this dataset.

Gap 3 — Nobody asks where the signal actually comes from. Papers compare models, but they do not ask which *kind* of information is doing the work — is it the clinical data, the demographics, or the patient’s past history?

1.4 My research problem

Given everything recorded about a hospital visit at the time the patient is discharged, predict the probability that the patient is readmitted within 30 days — using a testing method that respects the fact that the same patient appears many times, and reporting the answer as a probability a doctor could actually use.

1.5 My objectives and research questions

- **O1** — Build a clean, repeatable pipeline with no data leakage.
 - **RQ1** — How do logistic regression, random forest and boosting compare, and is the difference big enough to matter in practice?
 - **RQ2** — How much does the wrong splitting method inflate results?
 - **RQ3** — Do imbalance fixes help, or do they hurt?
 - **RQ4** — Which group of features carries the signal?
-

PART 2 — Dataset, EDA and models

2.1 The dataset and why I chose it

Diabetes 130-US Hospitals, 1999–2008. From the UCI Machine Learning Repository (dataset 296). 101,766 hospital visits, 71,518 patients, 130 hospitals, 50 columns. Free licence (CC BY 4.0).

Four reasons I chose it:

1. It is the only big free dataset that has **all six things** the question asks for in one table — demographics, diagnoses, lab results, procedures, medicines, and past admissions — plus a ready-made 30-day readmission label.
2. It has a **patient ID**. Without that I could not even ask RQ2.
3. It is widely used, so I can **compare my numbers with published papers**.
4. It has exactly the **problems the question mentions** — missing values, imbalanced outcome, lots of interacting variables.

Proof it is the real dataset (in data/CHECKSUM.txt): rows $\times$ columns $101,766 \times 50$, patients 71,518, classes NO 54,864 / >30 35,545 / <30 11,357 — all match the UCI description exactly. No synthetic data anywhere.

2.2 Cleaning

- Removed 2,423 visits where the patient **died or went to hospice**. They cannot be readmitted, so keeping them would be wrong.
- Removed 3 rows with invalid gender.
- **Left: 99,340 visits from 69,987 patients.**
- Target: 1 if readmitted in under 30 days, else 0. That is **11.4% positive**, so about **1 positive for every 8 negatives**.

I did **not** throw away repeat visits (many papers do). Those repeat visits are exactly where the signal is. I handled the problem at the splitting stage instead.

2.3 Missing values — the interesting part

Column	Missing	What I did
weight	96.9%	Dropped it
max_glu_serum	94.8%	Kept as “Not measured”
A1Cresult	83.1%	Kept as “Not measured”
medical_specialty	48.9%	Kept as “Missing”
payer_code	39.7%	Kept as “Missing”
race	2.2%	Kept as “Missing”

Why I did not just impute everything: I tested each column. I made a 0/1 column for “is this value missing” and ran a chi-square test against the outcome.

- For **weight**, missing means nothing ($p = 1.00$). So I dropped it.
- For **A1Cresult**, missing means **the doctor did not order the test** — and that is strongly linked to readmission ($p < 0.0000000001$). Patients with no HbA1c test are readmitted 11.7% of the time; patients with a normal result, 9.8%.

So the missingness itself is information. Imputing would have destroyed it. This is **Table 3** in my paper.

2.4 Feature engineering

I made **34 features** (19 numbers, 15 categories; 123 columns after one-hot encoding), in five groups:

Group	Examples
Demographic	age as a number, race, gender
Administrative	admission type, discharge destination, payer code, medical specialty
Clinical	ICD-9 codes grouped into 9 chapters, number of diagnoses, HbA1c, length of stay, tests per day
Medication	number of diabetes drugs, number of dose changes, insulin
Prior utilisation	past outpatient / emergency / inpatient visits, their total, count of the patient’s earlier visits

The main engineered ones: - **ICD-9 grouping** — raw codes like 250.83 are useless as categories (hundreds of them). I grouped them into 9 chapters (circulatory, respiratory, diabetes, etc.), following Strack’s paper. - **service_utilization** = outpatient + emergency + inpatient visits added together. - **A1C_tested / glucose_tested** — 0/1 flags for whether the test was ordered. - **prior_encounters** — how many earlier visits this patient already had.

2.5 EDA — what I found

I tested all 34 features against the outcome (chi-square for categories, Mann–Whitney for numbers). **30 out of 34 were significant** even after Bonferroni correction — but that is only because the sample is huge (about 100,000 rows). So I **ranked by effect size, not by p-value**. That is Figure 2.

The top 5 by effect size are **all past-history features**. Age is only 15th.

Key numbers to quote (Figure 3):

Finding	Numbers
Past inpatient visits	0 visits → 8.6% readmitted; 4+ visits → 31.4%
Discharge destination	transferred to another facility 16.4% vs sent home 9.2%
HbA1c	not measured 11.7% vs normal result 9.8%
Patient's own history	1st visit 9.0% → 6th visit or later 30.7%

2.6 How I split the data — the key decision

I split by **patient**, not by visit, using `GroupShuffleSplit` on the patient ID.

- Train: 74,583 visits from 52,490 patients
- Test: 24,757 visits from 17,497 patients
- **Patients appearing in both: zero**

For tuning I used `StratifiedGroupKFold` (3 folds) so the same rule applies inside cross-validation.

2.7 The models

Four model families, all tuned the same way (randomised search, 3 folds, scoring by average precision), plus a dummy baseline.

Model	AUROC	AUPRC	Sens@10%	PPV@10%
Baseline (predicts the average)	0.500	0.113	—	—
Logistic regression	0.664	0.206	21.2%	23.9%
Random forest	0.674	0.219	22.7%	25.6%
XGBoost	0.675	0.217	22.7%	25.6%
LightGBM (best)	0.677	0.223	23.5%	26.5%

Why AUPRC and not accuracy? With 11% positives, a model that says “nobody is readmitted” gets 89% accuracy and is useless. AUPRC has a no-skill value equal to the base rate — 0.113 here — so I can see how much better than nothing I am. I got 0.223, which is about double.

What this means in practice: if the hospital follows up the riskiest 10% of patients, my model catches 23.5% of all readmissions. The team reviews 3.8 patients for each readmission found. Random selection would only catch 10%. So it roughly doubles the hit rate.

Is LightGBM really better than logistic regression? Yes, but only just: +0.017 AUPRC (95% CI +0.012 to +0.024, $p < 0.001$ by paired bootstrap). Statistically real, but too small to change a discharge decision. I say this honestly in the paper.

2.8 My four experiments

Experiment 1 (RQ2) — does the wrong split inflate results?

I redid everything with a normal random split. 35.8% of test patients also appeared in training. **But the results barely changed** — AUROC went up by only 0.0027 on average.

This is a **negative result** and I report it. My explanation: leakage helps when the model can memorise something it cannot otherwise see. Here the thing that makes a patient's visits similar — their past history — is *already a feature*. So the model does not need to recognise the patient.

I still recommend patient-level splitting, because it costs nothing and it is correct.

Experiment 2 (RQ3) — do imbalance fixes help?

Same LightGBM, six different treatments of the imbalance:

Treatment	AUPRC	Calibration error	Average predicted risk
Nothing	0.223	0.003	0.113
Class weights	0.223	0.343	0.456
SMOTE	0.218	0.009	0.119
SMOTE-NC	0.155	0.159	0.271
Undersampling	0.214	0.351	0.464
Nothing + recalibration	0.211	0.005	0.112

Read the last column. The true rate is **0.113**. Class weights and undersampling predict **0.46** — four times too high. A doctor told “this patient has a 46% chance” would be badly misled. And none of them improved the ranking.

Conclusion: doing nothing is the best option. This matches van den Goorbergh (2022) and Welvaars (2023).

One extra thing I noticed: plain SMOTE barely did anything. It creates fake patients by averaging between real ones, which produces impossible values like “0.4 of a category”. The model spots those and ignores them. When I used SMOTE-NC, which handles categories properly, it actually made things **worse** (0.155).

Experiment 3 (RQ4) — where does the signal come from?

I added the feature groups one at a time:

Features used	AUROC	AUPRC
Demographics only	0.510	0.117
+ Administrative	0.605	0.167
+ Clinical	0.625	0.172
+ Medication	0.632	0.176
+ Prior utilisation	0.676	0.223

Demographics alone are worthless — AUROC 0.510 is basically a coin toss. Past history adds more than everything else put together. Removing it costs 0.047 AUPRC; removing demographics costs 0.0002.

Experiment 4 — which individual features matter?

SHAP on the best model, and odds ratios from logistic regression. They agree, which is reassuring because the two models work completely differently.

Top: number of past inpatient visits, discharge to another facility (+38% odds), discharge home (−23% odds), the patient’s own prior visits, missing payer code (+19% odds).

The uncomfortable finding: payer code and medical specialty are among the strongest predictors. Those are administrative, not medical. The model may be learning how the health system treats different patients, not how sick they are. I flag this as a fairness risk in the discussion.

2.9 Limitations (I should say these before sir does)

1. Data is from 1999–2008, before the HRRP penalty scheme. Practice has changed.
 2. If a patient goes to a hospital outside the network, I cannot see it. So the true readmission rate is higher than 11.4%.
 3. `prior_encounters` is counted inside the dataset window; a real deployment would need a fixed lookback period.
 4. Modest tuning budget, three folds.
 5. I did not do a full fairness audit by subgroup.
 6. One train/test split only; repeated splits would give tighter confidence intervals.
-

PART 3 — The paper

File: `paper/main.pdf` — 24 pages, Springer LNCS format.

- Official Springer `lncs.cls` version 2.25, taken from the LNCS Overleaf template. Times Roman font (`newtxtext`), as Springer requires.
 - Sections: Introduction, Related Work, Research Problem and Objectives, Data and Methods, Exploratory Analysis, Results, Discussion, Conclusion, References, Appendix.
 - **9 figures, 10 tables, 26 references** with DOIs, using the official `splncs04.bst` bibliography style.
 - **Every number and every figure is generated by the code.** The tables in the paper are `\input` from files that `src/analysis.py` writes. I never typed a result by hand, so the paper cannot disagree with the analysis.
-

What to show sir, in order

#	What	File
1	The literature search code	<code>lit/lit_search.ipynb</code> (open in Colab)
2	The search log and screening record	Table 1 and Appendix A of the paper
3	The raw dataset + proof it is genuine	<code>data/diabetic_data.csv</code> , <code>data/CHECKSUM.txt</code>
4	The full analysis notebook	<code>notebook/readmission_analysis.ipynb</code>
5	The paper	<code>paper/main.pdf</code>
6	The Overleaf project	<code>submission/overleaf_project.zip</code>
7	AI use declaration	<code>submission/prompt_history.pdf</code>

Numbers I must remember

- Dataset: **101,766 visits, 71,518 patients, 50 columns**
- After cleaning: **99,340 visits, 69,987 patients**
- Readmission rate: **11.4%**, imbalance about **1 to 8**
- Features: **34** (123 after encoding), in **5 groups**
- Split: **74,583 train / 24,757 test**, zero patient overlap

- Best model: **LightGBM, AUROC 0.677, AUPRC 0.223** (no-skill 0.113)
 - At a 10% alert rate: catches **23.5%** of readmissions, **3.8** patients reviewed each
 - Papers: **100 retrieved → 77 after dedup → 23 included**
 - Published benchmark: median AUROC **0.68** — so my result is exactly on the mark
-

Questions sir may ask, and my answers

“Your AUROC is only 0.68. Is that not poor?” It matches the published median of 0.68 across 41 studies (Mahmoudi, BMJ 2020). Twenty years of research sits in the same range. That is my finding: the limit is the **data**, not the algorithm. The things that really cause readmission — social support, whether someone takes their medicines, housing, follow-up appointments — are simply not in hospital records.

“Why not deep learning?” Liu (2024) tried an LSTM on this exact dataset and tree models still beat it. On 100,000 rows of tabular data, boosting is the right tool. Deep learning needs sequence or text data, which this dataset does not have.

“What is AUPRC and why use it?” Area under the precision–recall curve. With only 11% positives, accuracy and AUROC look good even for a bad model. AUPRC starts at the base rate — 0.113 here — so anything above that is real skill. I got 0.223.

“What is calibration and why does it matter?” Calibration means that when the model says 20%, about 20 out of 100 such patients actually come back. You need it if a doctor is going to act on the number. My best model is well calibrated (error 0.003). The moment I applied class weights, it started saying 46% when the truth was 11%.

“What is new in your work?” Three things. I **measured** the leakage everyone warns about and found it is small — that is a new number for this dataset. I **tested** imbalance corrections on ranking and calibration together and showed they only hurt. And I **separated** the feature groups to show that demographics contribute nothing while past history contributes almost everything.

“You got a negative result on leakage. Is that a failure?” No. I expected leakage to matter, I measured it, and it did not. Reporting that honestly is better science than hiding it. I explain *why* in the paper: the model does not need to recognise the patient because the patient’s history is already a feature.

“Where did your references come from? Did you make them up?” No. They came out of the OpenAlex API through my notebook, with real DOIs, and I checked the author lists against the publishers. The full search log with dates and hit counts is Table 1, and every one of the 77 candidates with my decision is in Appendix A.

“Is any of this data synthetic?” No. The pipeline reads exactly one file, the raw UCI CSV. Its SHA-256 checksum, row count, patient count and class counts are in `data/CHECKSUM.txt` and match the UCI release exactly. The word “synthetic” appears twice in the paper and both times it means SMOTE — Synthetic Minority Over-sampling Technique — which is a method I tested and found harmful, not data I created.

Words I should be able to explain in one line

- **AUROC** — chance the model ranks a readmitted patient above a non-readmitted one.
- **AUPRC** — same idea but focused on the rare class; starts at the base rate.
- **Calibration** — does “20%” really mean 20%?
- **Brier score / ECE** — two ways of measuring how far off the probabilities are.

- **Class imbalance** — one outcome is much rarer than the other (11% vs 89% here).
- **SMOTE** — makes fake minority-class rows by averaging real ones.
- **Data leakage** — test information sneaking into training; here, the same patient on both sides.
- **GroupShuffleSplit** — splits by a group (my patient ID) instead of by row.
- **SHAP** — tells you how much each feature pushed one prediction up or down.
- **Decision curve / net benefit** — is using the model better than treating everyone or treating nobody?